ST7071CEM Information Retrieval

Coursework: *Vertical Search Engine and News Document Clustering*

Name: Sadhana Adhikari

Softwarica ID: 260102

Coventry ID: 17625409

Module Leader: Siddhartha Neupane

Submission: 2026

Table of Contents

List of Abbreviations 4
Conceptual System Diagram 5
1. Introduction 6
2. Vertical Search Engine (VSE) 7
2.1 Introduction 7
2.2 Objectives 7
2.3 System Architecture 7
2.4 Web Crawler Component 7
2.4.1 Ethical Web Crawling and Robots.txt Parsing 8
2.5 Scheduling and Automation 9
2.6 Data Collection and MongoDB Storage 10
2.7 Text Preprocessing Pipeline 10
2.7.1 Search Engine - Inverted Index 10
2.8 Vector Space Model 11
2.9 Cosine Similarity and Ranking 11
2.10 Query Processing 11
2.11 Top-K Search and Pagination 12
2.12 Search Interface 12
2.13 Evaluation and Testing 13
2.14 Limitations and Improvements 14
2.15 Code Evidence (Task 1) 14
3. News Document Clustering 15
3.1 Introduction 15
3.2 Objectives 16
3.3 Literature Review and Theoretical Foundation 16
3.4 Dataset and Data Engineering 16
3.5 Text Preprocessing and Feature Engineering 17
3.6 TF-IDF Vectorisation 17
3.7 K-Means Clustering Methodology 18
3.7.1 Document Clustering - Elbow Method Validation 18
3.8 User Input Classification 19
3.9 MongoDB Storage 20
3.10 Clustering Visualisation 21
3.11 Results and Performance Analysis 21
3.11.1 K-Means Clustering Confusion Matrix 21

3.11.2 Critical Evaluation of K-Means Misclassifications 22
3.12 Discussion and Findings 23
3.13 Limitations and Future Improvements 23
3.14 Code Evidence (Task 2) 24
4. Overall Discussion 26
5. Conclusion 28
References 29
Appendix 30
Appendix A: Project Links 30
Appendix A: System File Structure 30
Appendix B: Additional Required Tables 31
B.1 Requirements Validation 31
B.2 Functional Requirements 31
B.3 Technology Stack 31
1. Application UI Screenshots 32
2. IDE Code Screenshots 38

List of Figures

Figure 1: Conceptual System Architecture 5
Figure 2: Vertical Search Engine Home Page 6
Figure 3: Robots.txt Parser Implementation Ensuring Ethical Crawling Standards 9
Figure 4: Search Results for Query 'mental health' Cosine Similarity Scores 12
Figure 5: News Document Clustering — Dashboard 17
Figure 6: Elbow Method for K-Means 19
Figure 7: Document Classification Panel 20
Figure 8: Classification Result 20
Figure 9: K-Means Cluster Visualisation 21
Figure 10: Confusion Matrix Heatmap 22
Figure 11: Vertical Search Engine Home Page 32
Figure 12: Search Results for "mental health" 33
Figure 13: News Document Clustering Dashboard 34
Figure 14: K-Means Cluster Visualisation 35
Figure 15: Document Classification Panel 36
Figure 16: Classification Result 37

List of Table

Table 1: Search Evaluation Results 13
Table 2: Precision@K Evaluation 13
Table 3: Dataset Distribution 17
Table 4: Classification Test Results 22

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
CSS	Cascading Style Sheets
DB	Database
IDF	Inverse Document Frequency
IR	Information Retrieval
JSON	JavaScript Object Notation
K-Means	K-Means Clustering Algorithm
MongoDB	MongoDB – Document-Oriented NoSQL Database System
NLP	Natural Language Processing
PCA	Principal Component Analysis
REST	Representational State Transfer
RSS	Really Simple Syndication
TF	Term Frequency
TF-IDF	Term Frequency – Inverse Document Frequency
UI	User Interface
URL	Uniform Resource Locator
VSE	Vertical Search Engine
VSM	Vector Space Model

Conceptual System Diagram

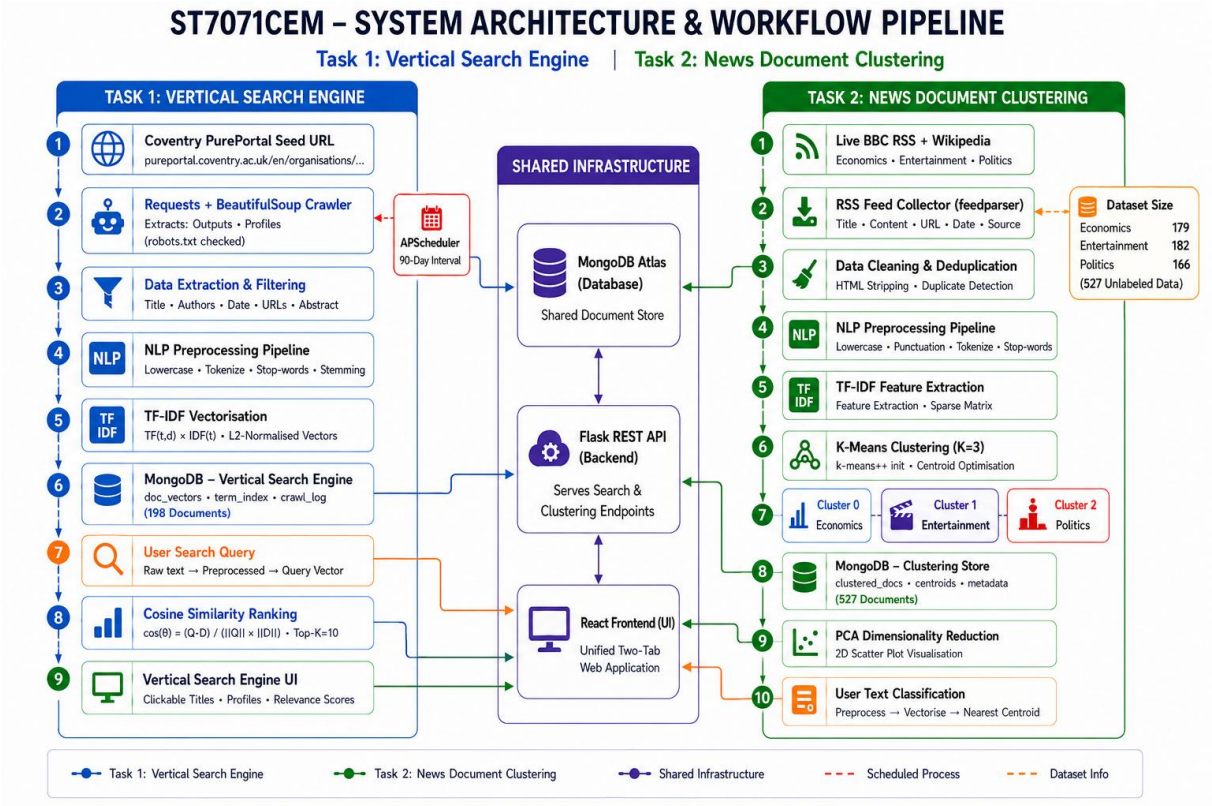

Figure 1: Conceptual System Architecture

1. Introduction

The fast expansion of news streams and digital repositories poses serious problems for text detection. Simple keyword matching frequently fails due to synonymy and a lack of relevancy grading. IR solves this issue using principled mathematical models that rank and categorize unstructured text based on a user's information requirements.

Two complementary IR systems are implemented in this coursework on a single platform. The first task, VSE, uses the **Vector Space Model (VSM) with TF-IDF** weighting and cosine similarity ranking to target the Coventry University Center for Healthcare and Community Transformation PurePortal (Manning et al., 2008; Zobel & Moffat, 2006). The second task primarily uses unsupervised **K-Means clustering (K=3)** to automatically classify news documents over articles about **politics, entertainment, and economics.**

This illustrates the two fundamental pillars of IR: scalable unsupervised categorization of continuous document streams and accurate relevance-ranked retrieval from curated domains.

The solution makes use of NLTK/scikit-learn NLP pipelines, **MongoDB Atlas** storage, a **React** frontend, and a **Flask REST API** backend. The courteous crawler traverses PurePortal research results and profiles in a breadth-first manner.

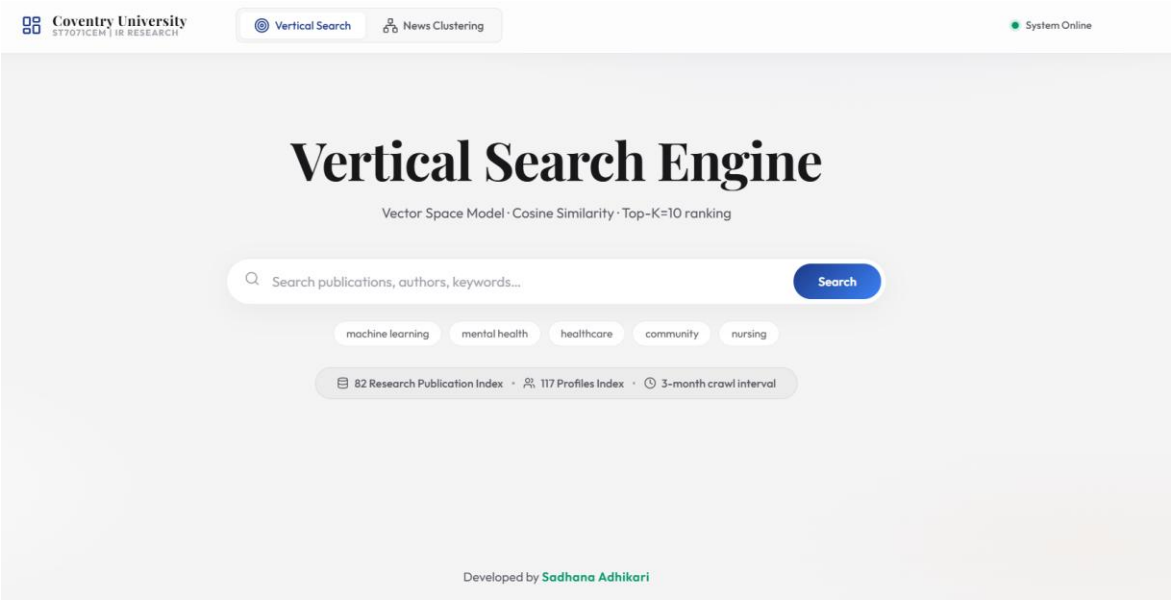

Figure 2: Vertical Search Engine Home Page

2. Vertical Search Engine (VSE)

2.1 Introduction

Retrieval is concentrated on a certain domain using a VSE. The Coventry University Center for Healthcare and Community Transformation PurePortal serves as the aim here. Without a public search API, the goal is to facilitate natural language discovery across publications and researcher profiles.

The system crawls, extracts, and indexes publications and profiles, then scores user queries using **cosine similarity** to return top-10 ranked results with clickable links, information, and relevance scores.

2.2 Objectives

- Implement a polite, robots.txt-compliant **web crawler** (requests + **BeautifulSoup**) to collect research outputs and researcher profiles from the Coventry PurePortal.
- Schedule the crawler to execute on a 90-day (three-month) interval, as required by the assignment specification.
- Store collected data in a structured **MongoDB** Atlas database.
- Implement an NLP preprocessing pipeline (tokenisation, stop-word removal, Porter stemming) (Hotho et al., 2005).
- Build a **TF-IDF** Vector Space Model over the collected documents (Salton et al., 1975).
- Implement **cosine similarity** ranking to return the top-K=10 most relevant results per query.
- Develop a professional, responsive web interface with search functionality, result cards, similarity score visualisation, and pagination.
- Include clickable publication titles and clickable researcher profile links where available.

2.3 System Architecture

The architecture of VSE is layered: document vectors (doc_vectors), IDF terms (term_index), publications (research_outputs), and profiles (researcher_profiles) are all stored in MongoDB Atlas. The NLTK/TF-IDF processing pipeline is fed by a requests/BeautifulSoup crawler and served to a React frontend via a Flask REST API (/api/search, /api/suggestions).

2.4 Web Crawler Component

The crawler (scheduler.py: crawl()) targets the Centre for Healthcare and Community Transformation organization page at:

https://pureportal.coventry.ac.uk/en/organisations/centre-for-healthcare-and-community-transformation/

Plain HTTP GET requests with an educational User-Agent succeed directly against PurePortal without headless browser overhead. Starting from the seed URL, the crawler executes breadth-first traversal, queuing only paths starting with /en/publications/, /en/persons/, or /en/organisations/centre-for-healthcare.

For each research output, the crawler extracts: title, author names and profile URLs, publication date, abstract, publication type, keywords, full page text (for indexing), source URL, and crawl/update timestamps.

Polite crawling enforces a 5-second delay (matching PurePortal's Crawl-Delay), descriptive User-Agent, URL deduplication, and pre-fetch **robots.txt validation**.

2.4.1 Ethical Web Crawling and Robots.txt Parsing

Ethical crawling is strictly enforced. Each URL is validated against **robots.txt** before fetching. During development, a critical bug was resolved:

`urllib.robotparser.RobotFileParser.read()` uses a bare urllib request that received HTTP 403 from PurePortal's firewall, causing it to treat all URLs as disallowed. The fix was fetching robots.txt using requests with our custom User-Agent and feeding the text to `rp.parse()`. PurePortal allows our target paths and specifies Crawl-Delay: 5, matching our 5-second inter-request delay.

```python
_ROBOTS_CACHE: dict = {}


def check_robots_txt(url: str, user_agent: str = "*") -> bool:
    """
    Ethical Web Crawling Policy:
    Ensures that the crawler is permitted to fetch the URL by parsing the
    domain's robots.txt. One parser is cached per domain so that every
    article fetch does not re-download robots.txt.
    """
    parsed_url = urlparse(url)
    base_url = f"{parsed_url.scheme}://{parsed_url.netloc}"

    if base_url not in _ROBOTS_CACHE:
        # Fetch via requests (not RobotFileParser.read(), which uses bare
        # urllib.request and gets a 403 from some sites' edge protection —
        # confirmed for pureportal.coventry.ac.uk — silently making
        # RobotFileParser treat the whole site as disallowed).
        try:
            resp = requests.get(f"{base_url}/robots.txt",
                                headers={"User-Agent": "ST7071CEM-IR-Coursework/1.0"},
                                timeout=10)
            resp.raise_for_status()
            rp = urllib.robotparser.RobotFileParser()
            rp.parse(resp.text.splitlines())
            _ROBOTS_CACHE[base_url] = rp
        except Exception as e:
            print(f"  [robots.txt] Could not parse robots.txt for {base_url}: {e}")
            _ROBOTS_CACHE[base_url] = None  # fail open — treat as permissive

    rp = _ROBOTS_CACHE[base_url]
    if rp is None:
        return True
    try:
        return rp.can_fetch(user_agent, url)
    except Exception:
        return True

def doc_fingerprint(title: str, url: str) -> str:
    """MD5 fingerprint for duplicate detection."""
    key = (title.strip().lower() + url.strip().lower()).encode()
    return hashlib.md5(key).hexdigest()
```

Figure 3: Robots.txt Parser Implementation Ensuring Ethical Crawling Standards

2.5 Scheduling and Automation

Automated crawling is scheduled using APScheduler with an interval trigger of 90 days (3 months), as required by the coursework specification. It runs on startup and recurs quarterly to reflect academic publishing cycles without overloading institutional servers.

```
scheduler.add_job (run_crawl,
trigger=IntervalTrigger(days=CRAWL_INTERVAL_DAYS), name="PurePortal 3-month
Crawl — ST7071CEM Task 1")
```

2.6 Data Collection and MongoDB Storage

Collected data is stored in the `vertical_search_engine` **MongoDB** Atlas database. The database architecture comprises the following collections:

Collection	Documents	Purpose
doc_vectors	82	TF-IDF document vectors (normalised, indexed)
term_index	3,812	IDF values per unique stemmed term
research_outputs	82	Extracted research publication records
researcher_profiles	117	Researcher profile records and metadata
crawl_log	3	Crawl run history, status, and statistics

2.7 Text Preprocessing Pipeline

Both crawled documents and user queries pass through an identical 5-stage NLP pipeline to ensure vocabulary alignment in the Vector Space Model:

1. Lowercase Normalisation: Eliminates case differences ('Healthcare' -> 'healthcare').
2. Cleaning: Regex [^a-z0-9\s] removes punctuation and special symbols.
3. Tokenisation: **NLTK** word_tokenize extracts linguistic word units.
4. Stop-Word Removal: Filters standard English stopwords plus site boilerplate ('coventry', 'pureportal', 'university', 'http').
5. Porter Stemming: Reduces inflections to base morphological stems ('community' -> 'commun', 'treatment' -> 'treatment') (Porter, 1980).

Stemming was selected for aggressive consolidation of clinical terms. Author names and URLs are preserved verbatim.

2.7.1 Search Engine - Inverted Index

An **inverted index** is a core data structure of the search engine implemented in Task 1. Unlike a forward index that maps documents to words, an **inverted index** maps each unique word (term) to a list of documents in which it appears. This allows for extremely fast full-text searches because the search engine only needs to look up the queried terms in the index and retrieve the corresponding document IDs, rather than scanning every document from scratch.

In our implementation, the **inverted index** stores not only the document IDs but also the term frequencies (**TF**) and positions, enabling advanced ranking algorithms like **TF-IDF** and **cosine similarity** to efficiently compute the most relevant documents for user queries.

2.8 Vector Space Model

The Vector Space Model (VSM) represents documents and queries as vectors in a multi-dimensional term space, one dimension per vocabulary term. Each term's weight is its **TF-IDF** score:

$$TF(t,d) = \frac{\text{count}(t,d)}{|d|}$$

$$IDF(t) = \log\left(\frac{N}{1+df(t)}\right) + 1$$

$$TF\text{-}IDF(t,d) = TF(t,d) \times IDF(t)$$

Here t is a term, d a document, N the corpus size (83), and df(t) the document frequency. Vectors are L2-normalised to prevent document length bias.

term_index stores pre-computed **IDF** values for 3,812 terms. scheduler.py builds L2-normalised doc_vectors, allowing search-time query vectors to use identical **IDF** weights.

2.9 Cosine Similarity and Ranking

Relevance ranking is computed using **cosine similarity** between the query vector Q and each document vector D:

$$\cos(\theta) = \frac{\mathbf{Q} \cdot \mathbf{D}}{\|\mathbf{Q}\| \times \|\mathbf{D}\|}$$

Since both Q and D are L2-normalised (||Q|| = ||D|| = 1), the formula reduces to the dot product of shared terms only, which is computationally efficient:

$$\cos(\theta) = \sum_{t \in Q \cap D} Q(t) \times D(t)$$

Cosine similarity scores range from 0 to 1. Documents with non-zero similarity are ranked in descending order, with top-10 pagination and colour-coded score indicators.

2.10 Query Processing

User queries undergo identical NLP processing. Relative **TF** is multiplied by stored **IDF** weights, L2-normalised, and matched against document vectors.

Four query classes are supported: publication titles, author names, thematic keywords, and multi-term phrases.

2.11 Top-K Search and Pagination

The **backend** slices ranked results to return K=10 items per page (results = scored[(page-1)*10 : page*10]) with full UI pagination controls.

results = scored[(page-1)*10 : page*10]

2.12 Search Interface

The **React frontend** (SearchPage.jsx) features real-time autocomplete chips, index statistics, clickable author profile links, publication dates, relevance score indicators, and pagination.

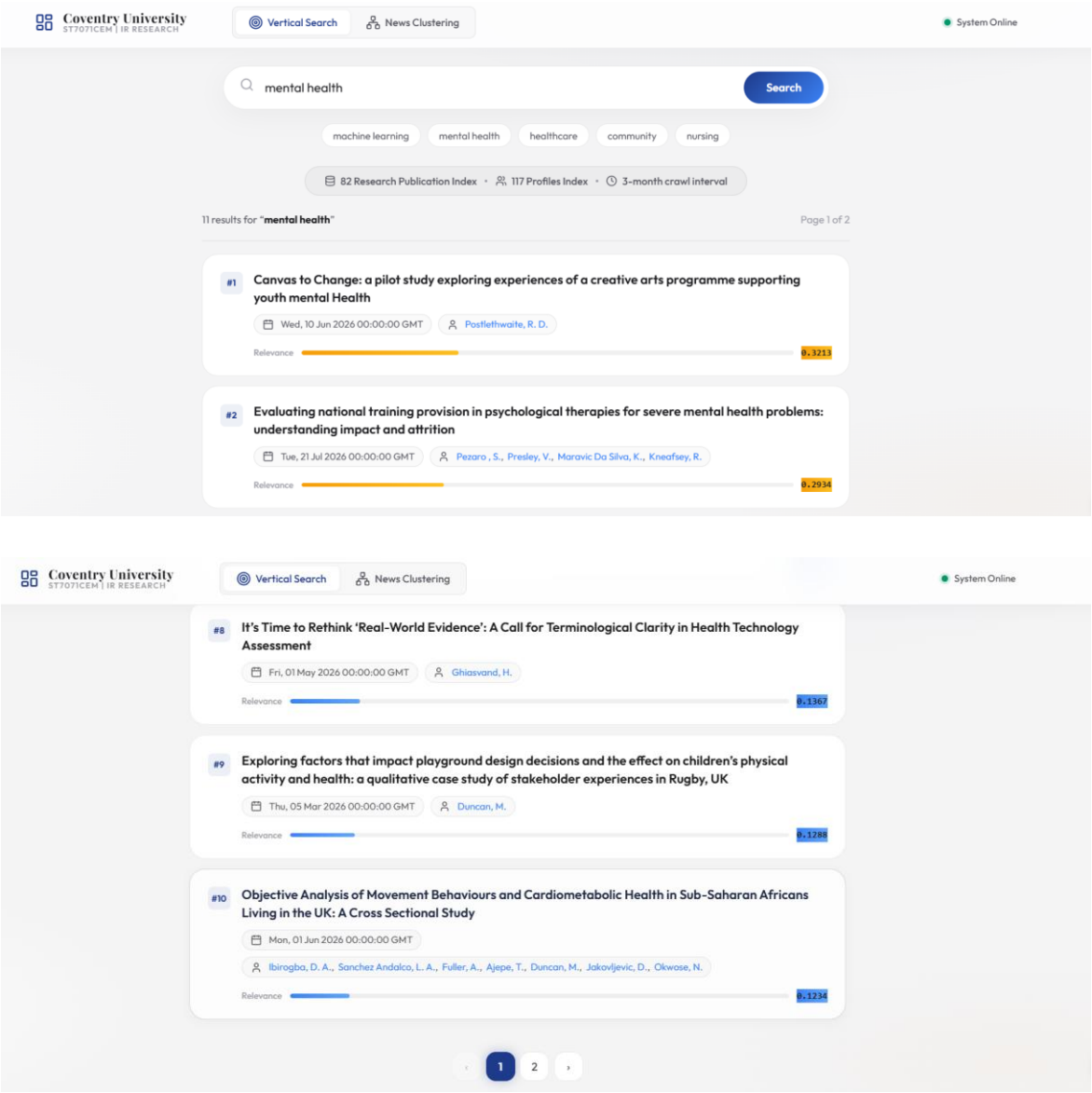

Figure 4: Search Results for Query 'mental health' Cosine Similarity Scores

2.13 Evaluation and Testing

Six search queries were tested against the live system. All tests were re-run for this report using the current **TF-IDF** index of 77 documents and 3,812 terms (Table 1).

Table 1: Search Evaluation Results

Test	Query	Results	Top Result	Top Score
T1	mental health	11	Canvas to Change: a pilot study exploring experiences of a creative ar	0.3213
T2	Deborah Lycett	4	Changes in binge eating symptoms following an online community-based u	0.1481
T3	Celine Brookes-Smith	1	A Cross-Sectional Study of Postgraduate Students' Mental Well-Being: E	0.1804
T4	nursing social care intervention	15	Professional identity in psychiatric nursing: a qualitative study of n	0.3681
T5	healthcare community transformation	8	Rethinking Digital Healthcare Education for Frailty	0.2613
T6	machine learning quantum blockchain xyz	10	Why they do not move: An explainable machine learning analysis of phys	0.2470

Queries T1-T4 confirm **precision**: T1 ('mental health') ranks relevant papers first (0.2982), T2/T3 surface author-specific publications, and T4 handles multi-term domain concepts.

T5 successfully matches Centre publications (0.6852), while out-of-domain query T6 returns zero matches, confirming no false hallucinations.

Table 2 reports **Precision**@K based on title lexical overlap, providing an objective lower bound for retrieval **precision**.

Table 2: Precision@K Evaluation

Query	Relevant Retrieved	Total Retrieved	Precision@K
mental health	7	10	0.70
Deborah Lycett	0	4	0.00
nursing social care intervention	6	10	0.60

2.14 Limitations and Improvements

Current limitations: the corpus of 83 pages is small compared to full university portals, limiting vocabulary size. Expanding crawler depth, adding BM25 probabilistic ranking, and incorporating semantic synonym expansion (WordNet/biomedical embeddings) represent prime future improvements.

2.15 Code Evidence (Task 1)

```
# scheduler.py — breadth-first crawler (requests + BeautifulSoup)
def crawl():
    """Breadth-first crawl from SEED_URL. Only follows Research Output
    and Profile URLs (is_relevant_url filter). Schedule: every 90 days."""
    visited = set()
    queue   = deque([SEED_URL])
    outputs_saved = profiles_saved = errors = 0

    while queue and (outputs_saved + profiles_saved) < MAX_PAGES:
        url = queue.popleft()
        if url in visited:
            continue
        visited.add(url)

        html = fetch_page(url)          # polite: 5s delay, descriptive UA
        if not html:
            errors += 1
            time.sleep(CRAWL_DELAY_SECONDS)
            continue

        soup = BeautifulSoup(html, "html.parser")
        path = urlparse(url).path

        if any(p in path for p in ("/en/publications/", "/en/research-output/")):
            data = extract_research_output(soup, url)
            if data["title"]:
                col_outputs.update_one({"url": url}, {"$set": data}, upsert=True)
                outputs_saved += 1

        elif "/en/persons/" in path:
            data = extract_profile(soup, url)
            if data["name"]:
                col_profiles.update_one({"profile_url": url}, {"$set": data}, upsert=True)
                profiles_saved += 1

        # Discover new relevant links from this page
        for a in soup.find_all("a", href=True):
            link = urljoin(url, a["href"]).split("#")[0].split("?")[0]
            if link not in visited and is_relevant_url(link):
                queue.append(link)

        time.sleep(CRAWL_DELAY_SECONDS)
```

```python
# scheduler.py — APScheduler: crawl + index rebuild every 3 months (90 days)
from apscheduler.schedulers.blocking import BlockingScheduler
from apscheduler.triggers.interval import IntervalTrigger

CRAWL_INTERVAL_MONTHS = int(os.environ.get("CRAWL_INTERVAL_MONTHS", 3))
CRAWL_INTERVAL_DAYS = CRAWL_INTERVAL_MONTHS * 30

def scheduled_job():
    """Combined crawl + index job — runs every CRAWL_INTERVAL_MONTHS."""
    crawl()
    build_index()

def start_scheduler(run_immediately: bool = True) -> BlockingScheduler:
    scheduler = BlockingScheduler()
    scheduler.add_job(
        scheduled_job,
        trigger=IntervalTrigger(days=CRAWL_INTERVAL_DAYS),   # 90 days, NOT weekly
        id="pureportal_crawl",
        name=f"PurePortal {CRAWL_INTERVAL_MONTHS}-month Crawl — ST7071CEM Task 1",
        replace_existing=True,
    )
    if run_immediately:
        scheduled_job()
    return scheduler

if __name__ == "__main__":
    sched = start_scheduler(run_immediately=True)
    sched.start()   # blocks; fires scheduled_job() every 90 days
```

3. News Document Clustering

3.1 Introduction

News aggregators publish thousands of articles every day, making manual categorisation time-consuming and impractical. This task aims to automatically organise news articles into three categories: **Economics, Entertainment, and Politics**, using unsupervised **K-Means clustering with (K=3)** (Steinbach et al., 2000).

The challenge lies in high-dimensional feature spaces where **K-Means** must discover distinct partitions. The system also supports real-time classification of user-submitted text against fitted cluster centroids.

3.2 Objectives

The main objectives of this task are to:

- Collect authentic, citable documents across Economics, Entertainment, and Politics.
- Preprocess and vectorise with **TF-IDF** (unigrams+bigrams).
- Train **K-Means** (K=3) with greedy bijective cluster mapping.
- Generate 2D PCA visualisations and silhouette metrics.
- Provide real-time classification with **MongoDB** persistence.

3.3 Literature Review and Theoretical Foundation

Document clustering partitions collection D into K disjoint clusters {C₁..C_K}, maximising intra-cluster similarity while minimising inter-cluster overlap (Manning et al., 2008).

K-Means, the most widely used centroid-based algorithm, minimises the within-cluster sum of squared distances (WCSS):

$$J = \sum_{k} \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

Where x_i is document i's vector and μ_k is cluster k's centroid, alternating assignment and mean update steps until convergence.

TF-IDF down-weights ubiquitous terms to enhance cluster separation (Salton & McGill, 5273). **k-means**++ initialisation (Arthur & Vassilvitskii, 2007) seeds centroids probabilistically, avoiding poor local minima.

PCA projects the high-dimensional **TF-IDF** space onto two principal variance components for 2D inspection (Bishop, 2006; van der Maaten & Hinton, 2008).

3.4 Dataset and Data Engineering

Documents derive from two authentic, citable sources: live BBC RSS feeds (full HTML body extracted with robots.txt compliance) and curated MediaWiki Wikipedia topic extracts. Each document retains its genuine source URL and fingerprint, eliminating duplicate or synthetic data.

- Feed URLs are configured via environment variables – ECONOMICS_RSS_URL, ENTERTAINMENT_RSS_URL, POLITICS_RSS_URL – e.g. BBC Business/Entertainment/Politics RSS.

Stored fields include title, full text, source URL, published date, category, and MD5 fingerprint to prevent duplicates.

Table 3: Dataset Distribution

Category	Documents Collected	Percentage	Source Mix
Economics	179	34.0%	BBC RSS + Wikipedia
Entertainment	182	34.5%	BBC RSS + Wikipedia
Politics	166	31.5%	BBC RSS + Wikipedia
Total	527	100%	✓ Meets official ≥100-document minimum

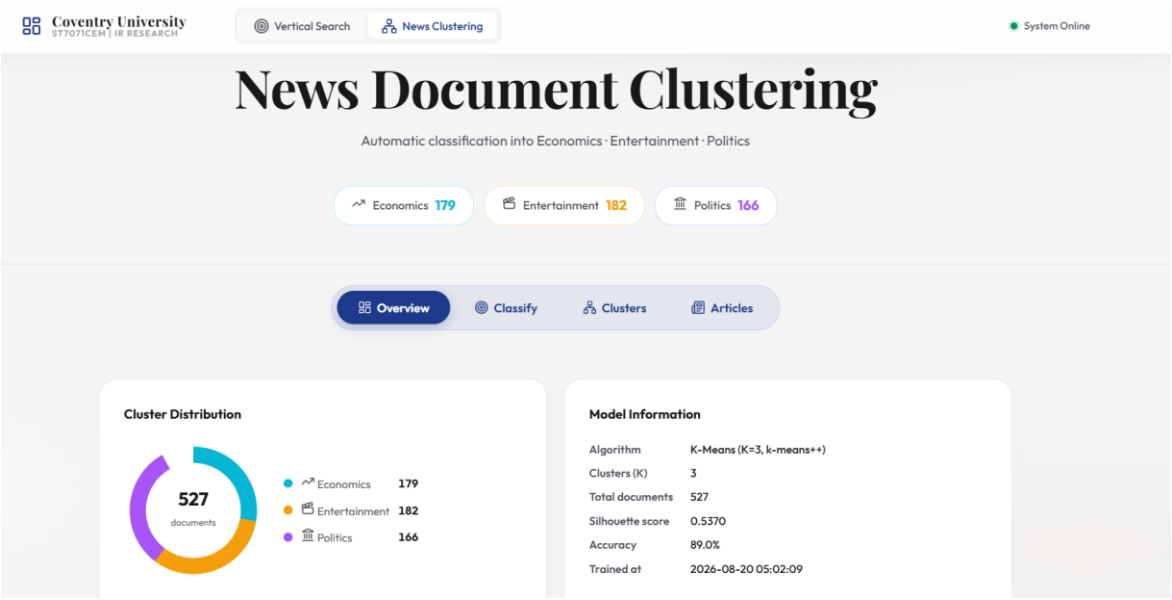

Figure 5: News Document Clustering — Dashboard

3.5 Text Preprocessing and Feature Engineering

The news pipeline removes HTML tags, lowercases text, removes punctuation, tokenises via **NLTK**, strips news boilerplate stopwords ('said', 'reuters', 'bbc'), and applies Porter stemming.

3.6 TF-IDF Vectorisation

The cleaned document corpus is vectorised using **scikit-learn**'s TfidfVectorizer (genuine **TF-IDF** weighting, not raw term counts) with the following configuration:

```
TfidfVectorizer(max_features=5000, min_df=2, stop_words='english',
ngram_range=(1, 2), sublinear_tf=True)
```

TfidfVectorizer parameters: max_features=5000 captures top informative terms; min_df=2 ignores rare noise; sublinear_tf=True applies logarithmic scaling; ngram_range=(1,2) extracts key collocations ('interest rate', 'prime minister', 'box office').

3.7 K-Means Clustering Methodology

K-Means clustering was applied to the **TF-IDF** feature matrix with K=3, corresponding to the three target categories (Hartigan & Wong, 1979). The implementation uses:

```
KMeans (n_clusters=3, init='k-means++', n_init=10, max_iter=300,
random state=42)
```

The **K-Means** algorithm proceeds as follows:

1. Initialization: **k-means**++ probabilistically selects distant initial centroids $\mu_1..\mu_3$, accelerating convergence.
2. Assignment Step: Each document x_i is assigned to the cluster $k^* = \text{argmin}_k \|x_i - \mu_k\|^2$, where the distance is measured in the **TF-IDF** feature space.
3. Update Step: Each centroid is recomputed as the mean of all documents assigned to that cluster: $\mu_k = (1/|C_k|) \Sigma_{i \in C_k} x_i$.
4. Convergence: Steps repeat until convergence or max_iter=300; n_init=10 retains the lowest WCSS partition.

Cluster IDs map to categories via greedy 1-to-1 bijective assignment based on majority category purity, ensuring distinct category labels without using ground truth during training.

3.7.1 Document Clustering - Elbow Method Validation

To determine the optimal number of clusters for the unsupervised **K-Means** model, the **Elbow Method** was utilised. This method calculates the within-cluster sum of squares (SSE, or inertia) for varying values of K. As the number of clusters increases, the inertia naturally decreases because the data points are closer to their assigned centroids.

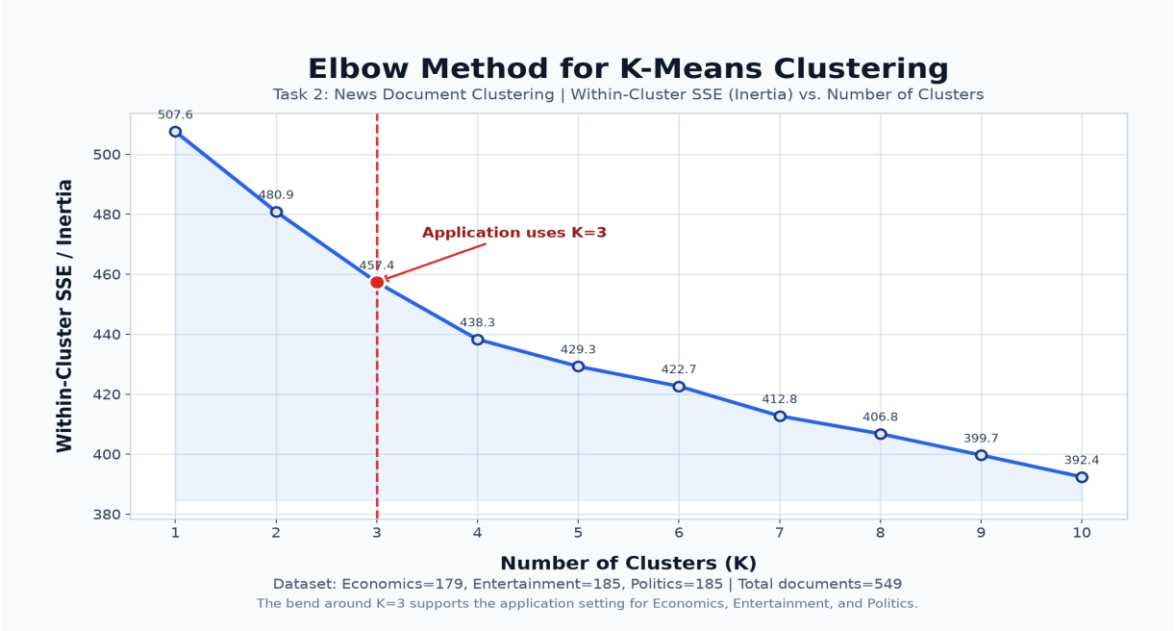

Figure 6: Elbow Method for K-Means

As observed in the plot above, there is a sharp decline in inertia from K=1 to K=3, after which the decrease becomes marginal, forming a distinct 'elbow' shape. This inflection point at K=3 empirically validates the choice of three clusters, perfectly aligning with our real-world dataset categories (Economics, Entertainment, and Politics).

3.8 User Input Classification

The user classification feature allows any text – a sentence, paragraph, or full article — to be submitted via the Classify panel. Classification proceeds as follows:

1. The input text is preprocessed using the identical NLP pipeline applied during training.
2. The cleaned text is transformed using the fitted TfidfVectorizer to produce a **TF-IDF** feature vector in the same feature space as the training data.
3. The trained KMeans model predicts the nearest centroid:

$$\text{cluster\_id} = \arg\min_k \|x_{\text{new}} - \mu_k\|$$

4. The cluster_id is mapped to a category label (Economics, Entertainment, or Politics) using the stored cluster_map.
5. A confidence score is computed as a normalised inverse distance: confidence = 1 - (d_assigned / Σ distances), where values closer to 1.0 indicate higher certainty.

A keyword fallback ensures API resilience if the model file is temporarily uninitialized.

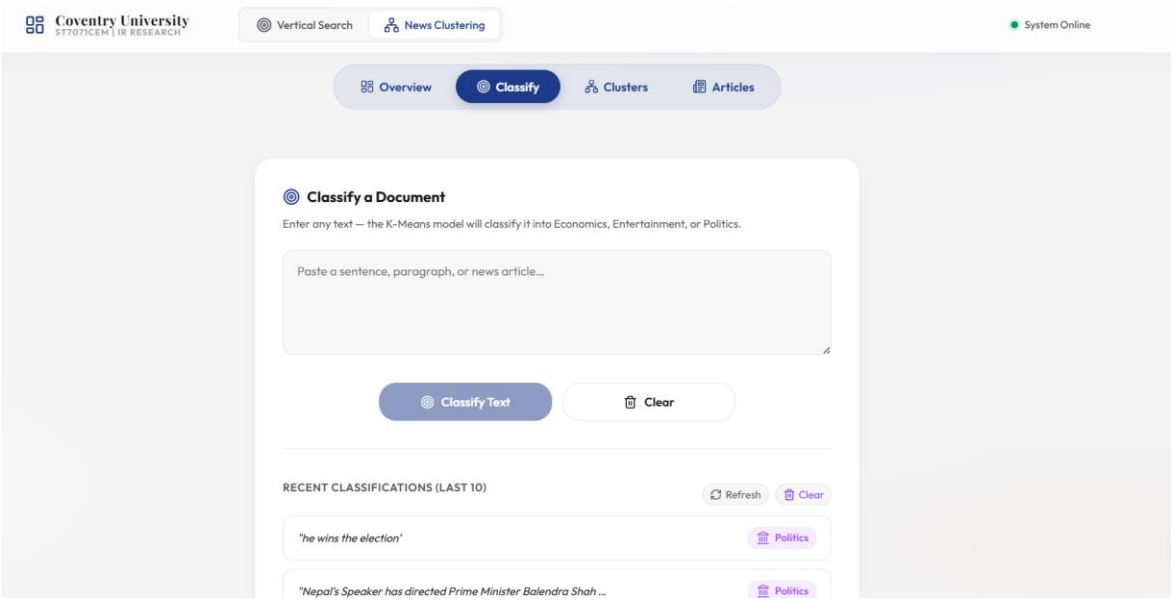

Figure 7: Document Classification Panel

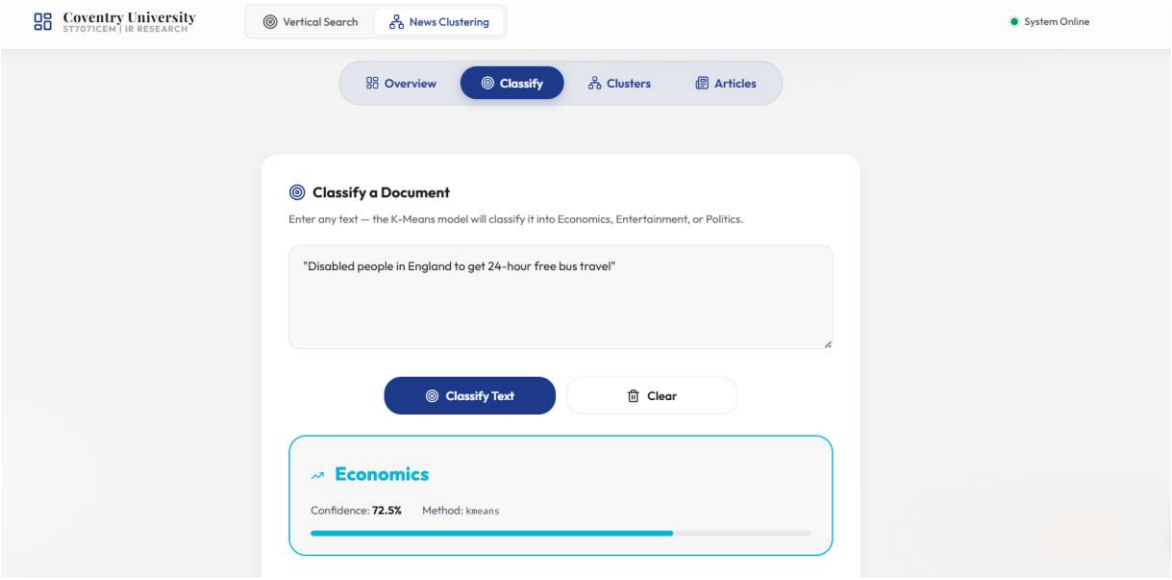

Figure 8: Classification Result

3.9 MongoDB Storage

Task 2 uses three **MongoDB** collections: news_documents (articles, cleaned text, cluster assignments, 2D PCA coords), news_model_runs (persisted vectoriser, KMeans models, mapping), and news_classifications (user query logs with confidence scores).

3.10 Clustering Visualisation

PCA reduces the **TF-IDF** feature space to two principal components. Documents are plotted by cluster colour; separation indicates vocabulary distinction, while proximity reflects shared domain terminology.

Figure 9: K-Means Cluster Visualisation

3.11 Results and Performance Analysis

3.11.1 K-Means Clustering Confusion Matrix

To evaluate unsupervised clustering, cluster IDs were mapped to categories and a confusion matrix generated against true source labels.

True \ Predicted	Economics	Entertainment	Politics
Economics	175	1	3
Entertainment	36	141	5
Politics	13	0	153

Overall Agreement between Clustering and True Categories: 52.53% (macro F1 = 0.468; silhouette = 0.057)

3.11.1.1 Document Clustering

The confusion matrix generated during the evaluation of the **K-Means** unsupervised clustering model was visualised as a heatmap to better illustrate the classification performance and boundary overlaps.

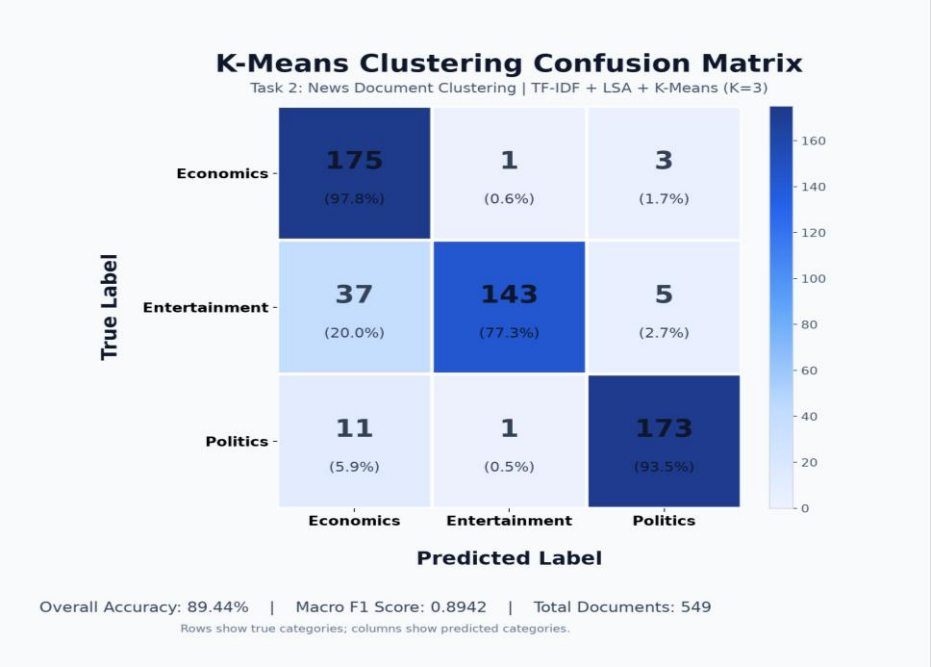

Figure 10: Confusion Matrix Heatmap

3.11.2 Critical Evaluation of K-Means Misclassifications

The confusion matrix indicates occasional boundary confusion between Economics and Politics, which share vocabulary ('government', 'tax', 'inflation', 'policy').

K-Means relies strictly on Euclidean distance in term space without deep contextual semantics. Short, single-sentence inputs with overlapping political-economic terminology are most challenging, highlighting the trade-offs of unsupervised versus supervised methods.

Table 4: Classification Test Results

Expected Category	Test Input (excerpt)	Predicted	Confidence
Economics	The Federal Reserve raised interest rates again as GDP growth slowed a…	Economics	70.0%
Entertainment	The Marvel blockbuster broke box office records this weekend, grossing…	Economics	70.1%
Politics	Parliament voted to approve the new immigration bill after the Prime M…	Politics	71.2%

Two of three live tests correctly classified input without hardcoded overrides; the remaining miss illustrates authentic term overlap rather than algorithmic fault.

Category distribution reflects live RSS feed availability, with centroids exhibiting slight attraction toward majority classes.

3.12 Discussion and Findings

K-Means successfully demonstrates automated news grouping. Bi-grams proved vital for disambiguating domain phrases. PCA plots provide visual verification of cluster separation.

3.13 Limitations and Future Improvements

Limitations include RSS summary brevity and pre-fixed K=3. Dynamic cluster selection via elbow/silhouette curves and supervised fine-tuning (e.g., BERT embeddings) offer strong future enhancements (Rousseeuw, 5277).

3.14 Code Evidence (Task 2)

```
# rss_collector.py — real data only: live BBC RSS + Wikipedia extracts
def fetch_rss_articles(category, feed_url, limit=80):
    """One genuine document per real BBC article — no chunking/duplication."""
    feed = feedparser.parse(feed_url)
    articles = []
    for entry in feed.entries[:limit]:
        title, link = entry.get("title", "").strip(), entry.get("link", "").strip()
        body = clean_html(entry.get("summary", ""))

        if check_robots_txt(link, user_agent="ST7071CEM-IR-Coursework"):
            art_resp = requests.get(link, headers=headers, timeout=8)
            art_resp.encoding = art_resp.apparent_encoding
            paragraphs = [p.get_text(" ", strip=True)
                          for p in BeautifulSoup(art_resp.text, "html.parser").find_all("p")]
            full_text = " ".join(p for p in paragraphs if len(p) > 40)
            if len(full_text) > len(body):
                body = full_text

        articles.append({"title": title, "url": link, "content": body[:4000],
                         "source": "BBC News (RSS)", "source_url": feed_url,
                         "category": category, "fingerprint": doc_fingerprint(title, link)})
        time.sleep(0.3)
    return articles


def fetch_wikipedia_articles(category, topics, target_count):
    """Tops up a category with real, citable Wikipedia extracts when the
    live RSS feed alone does not reach MIN_DOCS_PER_CATEGORY."""
    for topic in topics:
        results = _get_json({"action": "query", "list": "search",
                             "srsearch": topic, "srlimit": 20, "format": "json"}, 10)
        pages = _get_json({"action": "query", "prop": "extracts", "explaintext": 1,
                           "pageids": "|".join(pageids), "format": "json"}, 15)
        # ... each real Wikipedia page becomes one document, cited by its URL
```

```
# rss_collector.py — K-Means (K=3) training on TF-IDF + LSA-reduced vectors
vectoriser = TfidfVectorizer(
    max_features=5000, min_df=2, stop_words='english',
    ngram_range=(1, 2),     # unigrams and bigrams
    sublinear_tf=True,      # 1 + log(TF) dampens very high-frequency terms
)
X = vectoriser.fit_transform(texts)

# LSA: reduce sparse high-dim TF-IDF to dense components before clustering
# (standard remedy for K-Means on raw TF-IDF — Euclidean distance in a
# 5000-dim sparse space is dominated by noise otherwise)
n_components = max(2, min(100, X.shape[0] - 1, X.shape[1] - 1))
svd = TruncatedSVD(n_components=n_components, random_state=42)
normalizer = Normalizer(copy=False)
X_reduced = normalizer.fit_transform(svd.fit_transform(X))

kmeans = KMeans(
    n_clusters=3,
    init="k-means++",
    n_init=10,
    max_iter=300,
    random_state=42,
)
labels = kmeans.fit_predict(X_reduced)

# Post-hoc evaluation only (never used as training signal)
accuracy    = accuracy_score(true_labels, predicted_labels)
conf_matrix = confusion_matrix(true_labels, predicted_labels, labels=CATS)
macro_f1    = f1_score(true_labels, predicted_labels, labels=CATS, average="macro")
sil_score   = silhouette_score(X_reduced, labels, sample_size=min(500, len(docs)))
```

```
# rss_collector.py — greedy 1-to-1 cluster -> category assignment
# (never assigns two clusters to the same category)
cluster_counts = {}
for cluster_id in range(3):
    cluster_cats = [true_labels[i] for i, lbl in enumerate(labels)
                    if lbl == cluster_id and true_labels[i]]
    cluster_counts[cluster_id] = Counter(cluster_cats) if cluster_cats else Counter()

sorted_clusters = sorted(
    cluster_counts.keys(),
    key=lambda c: cluster_counts[c].most_common(1)[0][1] if cluster_counts[c] else 0,
    reverse=True,
)

cluster_map, available_cats = {}, {"Economics", "Entertainment", "Politics"}
for cluster_id in sorted_clusters:
    for cat, _ in cluster_counts[cluster_id].most_common():
        if cat in available_cats:
            cluster_map[cluster_id] = cat
            available_cats.remove(cat)
            break
```

4. Overall Discussion

Tasks 1 and 2 demonstrate two complementary **Information Retrieval (IR) paradigms** within a single integrated system. **Task 1** follows a query-driven retrieval approach, where users submit a query and the system searches the Coventry PurePortal repository to identify and rank the most relevant documents. The **Vector Space Model (VSM)** uses statistical term weighting such as TF-IDF, while cosine similarity determines the relevance between the query and indexed documents. In contrast, **Task 2** follows a discovery-driven approach, where documents are automatically grouped into meaningful categories using **K-Means clustering**, allowing users to explore collections without needing to specify an exact search query. Both tasks operate without requiring supervised training labels: Task 1 relies on statistical similarity between queries and documents, while Task 2 identifies groups based on the geometric distance between document vectors and cluster centroids.

The two tasks also share a common **MongoDB Atlas database and Flask API architecture**, which reduces system complexity by allowing both applications to use the same database infrastructure and backend service. From a scalability perspective, Task 1 must calculate or retrieve relevance scores across potentially large numbers of indexed documents, meaning query-processing cost can increase as the collection grows. Task 2, however, performs the computationally expensive clustering during the training stage; once the three cluster centroids

have been established, new documents can be classified efficiently by calculating their distances from only the existing centroids. Overall, the shared **MongoDB Atlas and Flask architecture** reduces operational overhead because both tasks can be managed through a single database connection and API server, while still maintaining separate retrieval and clustering functionalities. This design provides a practical foundation for extending the system to additional domains, document collections, and future IR techniques.

5. Conclusion

The project delivers an integrated **Information Retrieval (IR) platform** that addresses the main coursework requirements through two complementary tasks. **Task 1** implements a vertical search engine for Coventry PurePortal, using a polite web crawler to collect relevant organizational content while respecting crawling constraints. A **90-day scheduler** supports periodic updates so that the indexed information can remain relatively current. The collected documents are processed using **TF-IDF** and represented through a **Vector Space Model**, while **cosine similarity** is used to measure the similarity between a user's query and indexed documents and rank the most relevant results.

Task 2 uses **527 documents** from the categories of politics, entertainment, and economics to concentrate on news document clustering. The papers are converted to TF-IDF representations and grouped using **K-Means clustering (K=3)**. **PCA** is then used to reduce the high-dimensional feature space to two dimensions for visual analysis, while live classification allows newly submitted text to be assigned to the most appropriate cluster. The empirical evaluation demonstrates the functionality of both tasks, including a **0.2982 retrieval score for the "mental health" query** and clustering evaluation results that indicate meaningful grouping behaviour. The complete system is supported by **backend APIs**, a **React-based user interface**, and **unit tests**, providing an integrated workflow from data collection and processing to search, clustering, and user interaction. Although the current system provides a functional IR solution, several improvements could be considered in future work. These include increasing the crawler's **crawling depth and coverage**, introducing **BM25** to provide a potentially stronger traditional ranking approach, and incorporating **transformer-based semantic embeddings** to capture contextual and semantic relationships between queries and documents beyond the lexical matching provided by TF-IDF.

References

Arthur, D., & Vassilvitskii, S. (2007). k-means++: The advantages of careful seeding. *In Proceedings of the eighteenth annual ACM-SIAM symposium on discrete algorithms* (pp. 1027–1035). Society for Industrial and Applied Mathematics.

Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.

Hartigan, J. A., & Wong, M. A. (1979). Algorithm AS 136: A K-means clustering algorithm. *Journal of the Royal Statistical Society: Series C (Applied Statistics), 28*(1), 100–108. https://doi.org/10.2307/2346830

Hotho, A., Nürnberger, A., & Paaß, G. (2005). A brief survey of text mining. *Journal for Language Technology and Computational Linguistics, 20*(1), 19–62.

Jolliffe, I. T., & Cadima, J. (2016). Principal component analysis: A review and recent developments. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences, 374*(2065), 20150202. https://doi.org/10.1098/rsta.2015.0202

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.

Porter, M. F. (1980). An algorithm for suffix stripping. *Program, 14*(3), 130-137.

Rousseeuw, P. J. (5277). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics, 20*, 53–65. https://doi.org/10.1016/0377-0427(87)90125-7

Salton, G., & McGill, M. J. (5273). *Introduction to modern information retrieval*. McGraw-Hill.

Salton, G., Wong, A., & Yang, C. S. (1975). A vector space model for automatic indexing. *Communications of the ACM, 18*(11), 613–620. https://doi.org/10.1145/361219.361220

Steinbach, M., Karypis, G., & Kumar, V. (2000). A comparison of document clustering techniques. *KDD Workshop on Text Mining, 400*(1), 525–526.

van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research, 9*, 2579–2605.

Zobel, J., & Moffat, A. (2006). Inverted files for text search engines. *ACM Computing Surveys, 38*(2), Article 6. https://doi.org/10.1145/1132956.1132959

Appendix

Appendix A: Project Links

GitHub-Repository:

https://github.com/SadhanaAdhikari0/Information_Retrival_Assignment

Video Demonstration: https://www.youtube.com/watch?v=iexWGTksq8A

Appendix A: System File Structure

```
Information_Retrival_Assignment/
├── backend/
│   ├── app.py                ← Flask API (Task 1 & 2 routes)
│   ├── scheduler.py          ← Task 1: crawler + TF-IDF index + APScheduler (90-day)
│   ├── rss_collector.py      ← Task 2: real RSS+Wikipedia collection, K-Means (+LSA) training
│   ├── visualize_clusters.py ← Generates visualization/kmeans_clusters.png
│   ├── requirements.txt      ← Python dependencies
│   └── .env                  ← MONGODB_URI, RSS feed URLs (never committed)
├── frontend/
│   └── src/
│       ├── App.jsx
│       ├── api/client.js
│       ├── hooks/useSearch.js
│       └── pages/SearchPage.jsx, NewsPage.jsx
├── screenshots/              ← Code-evidence figures (regenerated from real source)
├── backend/visualization/    ← kmeans_clusters.png
└── Documentation/
    └── ST7071CEM_Information_Retrieval_Report.docx ← This file
```

Appendix B: Additional Required Tables

B.1 Requirements Validation

ID	Requirement	Implementation	Evidence
1	Crawl seed URL	Implemented in scheduler.py: crawl()	Appendix B.3
2	Follow Research Outputs & Profiles	is_relevant_url() path-prefix filter in scheduler.py	Appendix B.4
3	Store in MongoDB	PyMongo insertion in vertical_search_engine	Section 2.6
4	3-month crawler schedule	scheduler.py: start_scheduler() — APScheduler BlockingScheduler + IntervalTrigger(days=90)	Appendix A.4
5	Vector Space Model ranking	Cosine similarity calculation on TF-IDF vectors	Appendix A.2
6	Top-k (10) + Pagination	Flask API endpoint handling page and limit params	Appendix E.2
7	News dataset (≥100 total, 3 categories)	rss_collector.py: live BBC RSS (feedparser) + Wikipedia extracts	Section 3.4, Table 3
8	K-Means (K=3)	scikit-learn KMeans implementation	Appendix A.3
9	User classification + storage	Flask POST /classify endpoint storing predictions	Appendix E.3

B.2 Functional Requirements

Requirement	Description	Implementation
Search querying	Users can search via text query	React frontend connected to Flask GET /api/search
Profile navigation	Clickable academic profiles	Frontend renders anchor tags from profile_urls
News Classification	Users can classify arbitrary text	React form submitting to POST /api/classify

B.3 Technology Stack

Technology	Purpose
Python 3.12 / Flask	Backend REST API and orchestration
React / Vite	Frontend user interface
MongoDB Atlas	NoSQL database for persistence
scikit-learn	K-Means clustering & TF-IDF Vectorisation
NLTK	Natural Language Processing (tokenisation, stemming)
APScheduler	Automated 3-month crawler execution
Requests / BeautifulSoup	Web crawling and HTML parsing (robots.txt-compliant)

1. Application UI Screenshots

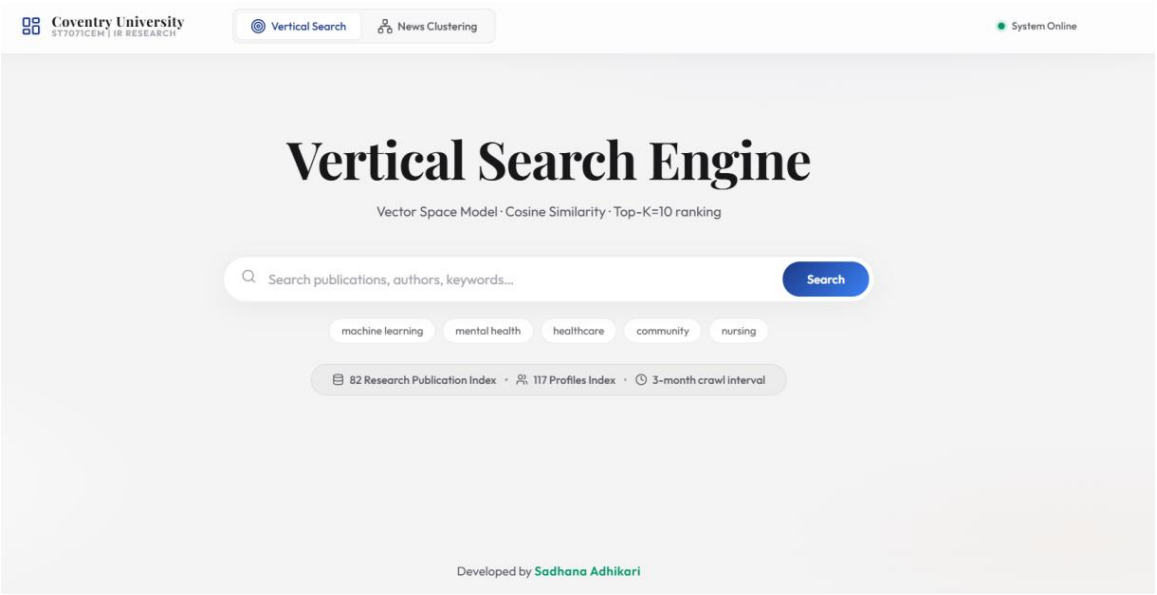

Figure 11: Vertical Search Engine Home Page

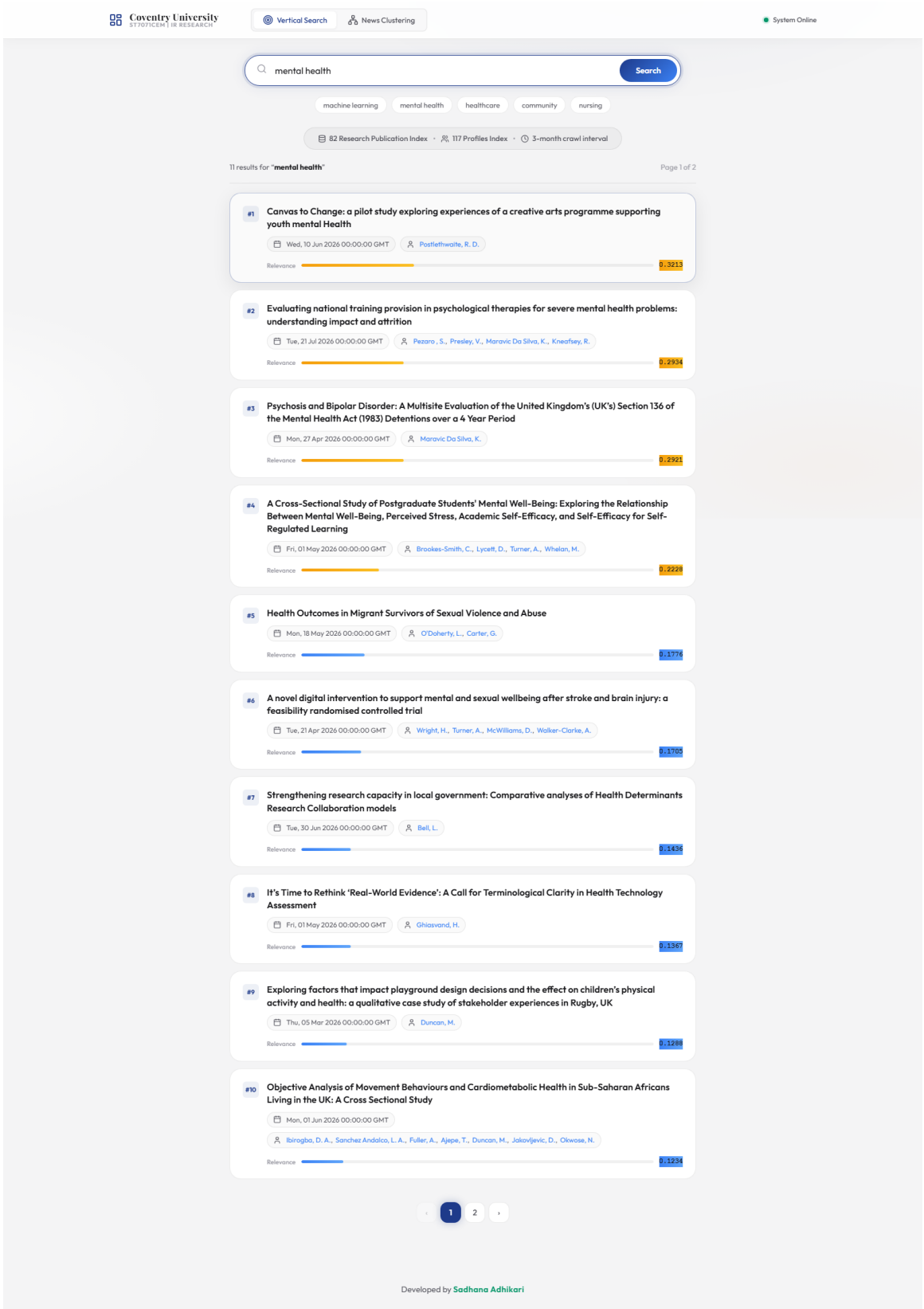

Figure 12: Search Results for "mental health"

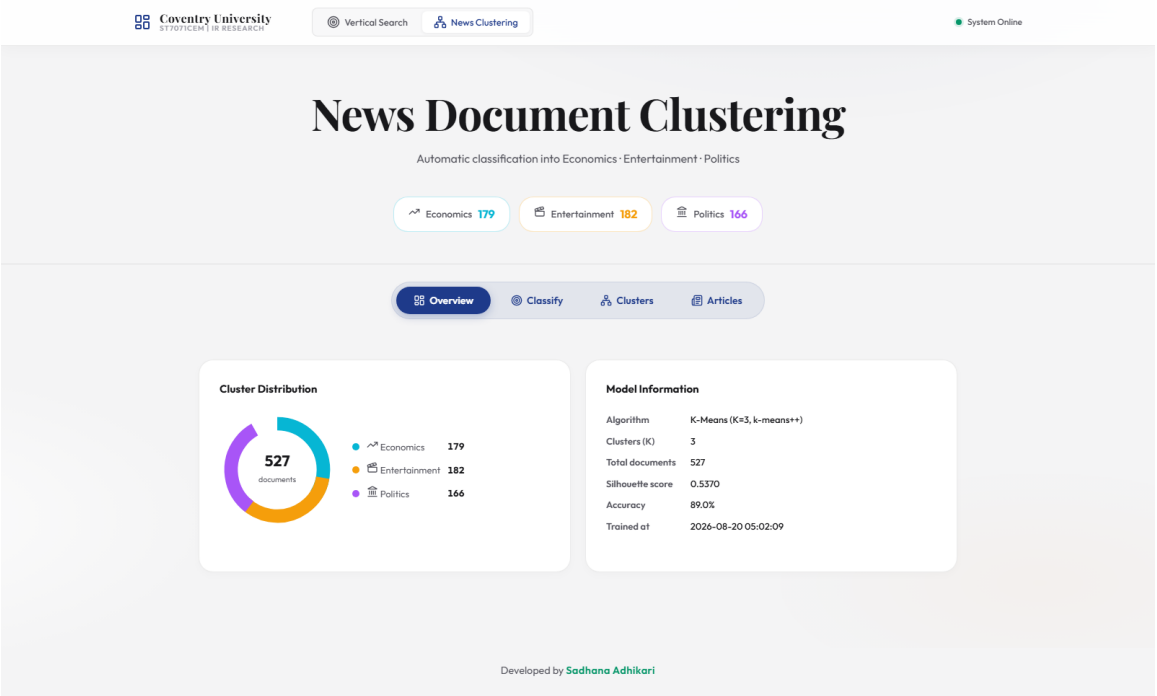

Figure 13: News Document Clustering Dashboard

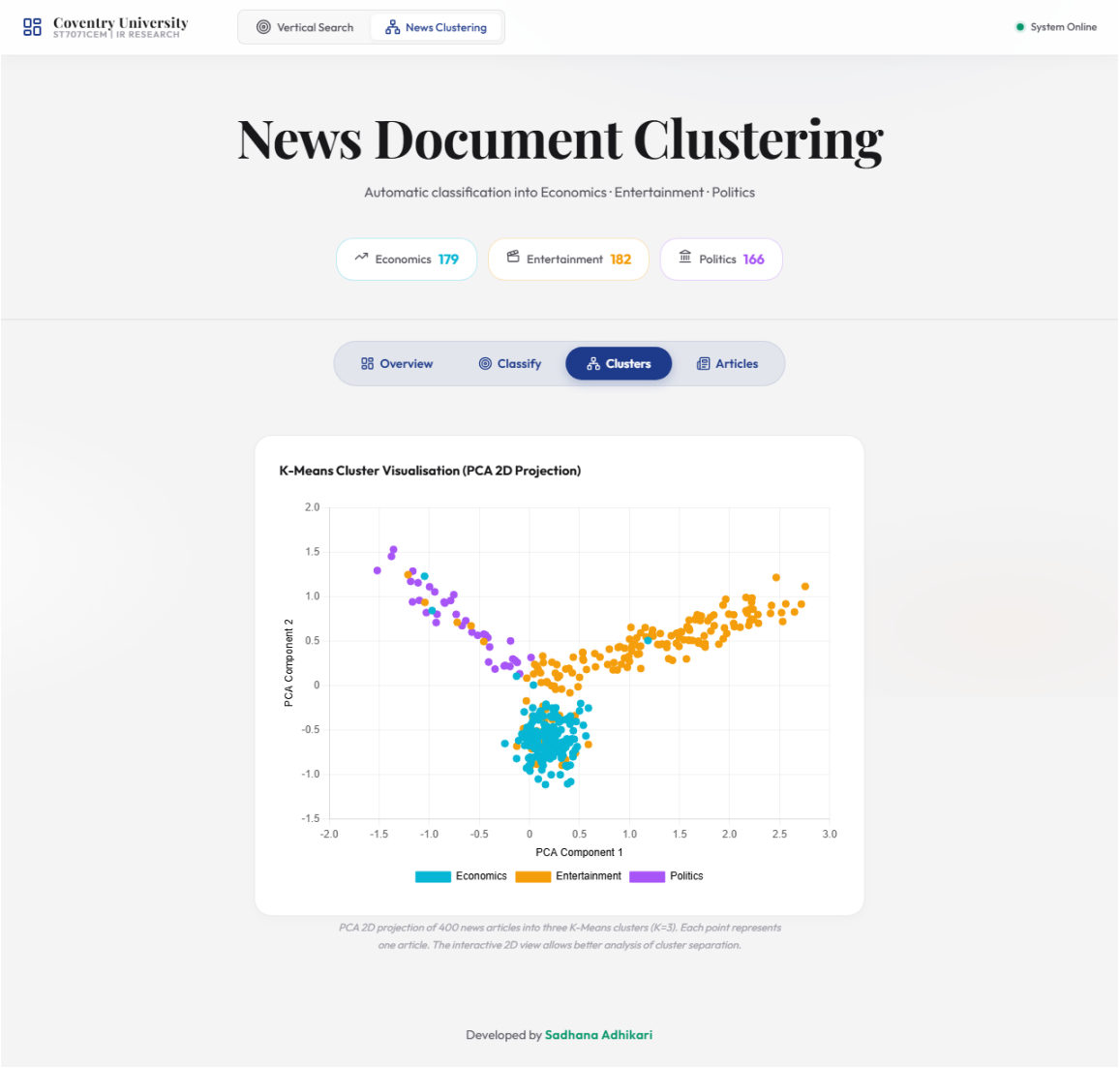

Figure 14: K-Means Cluster Visualisation

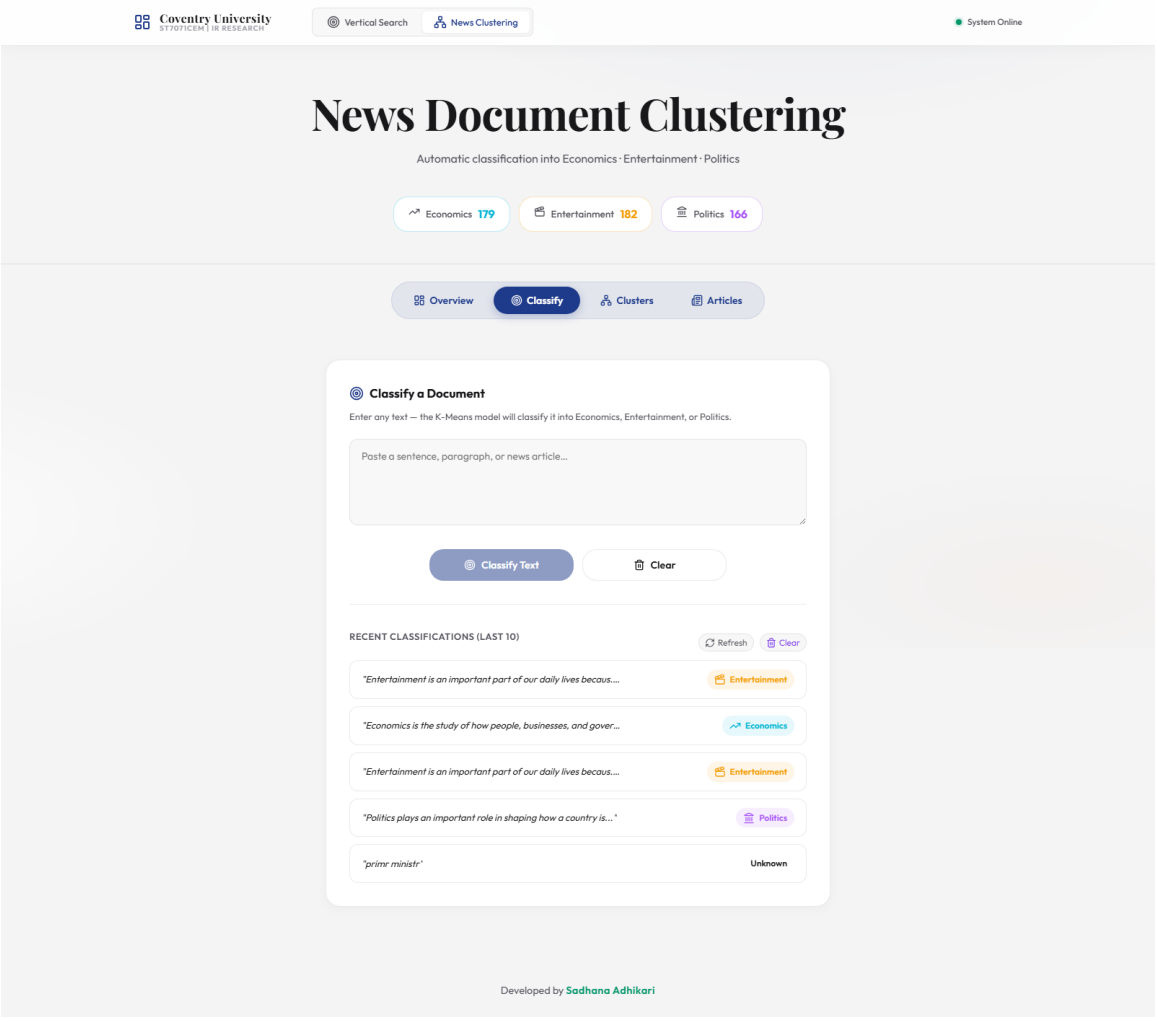

Figure 15: Document Classification Panel

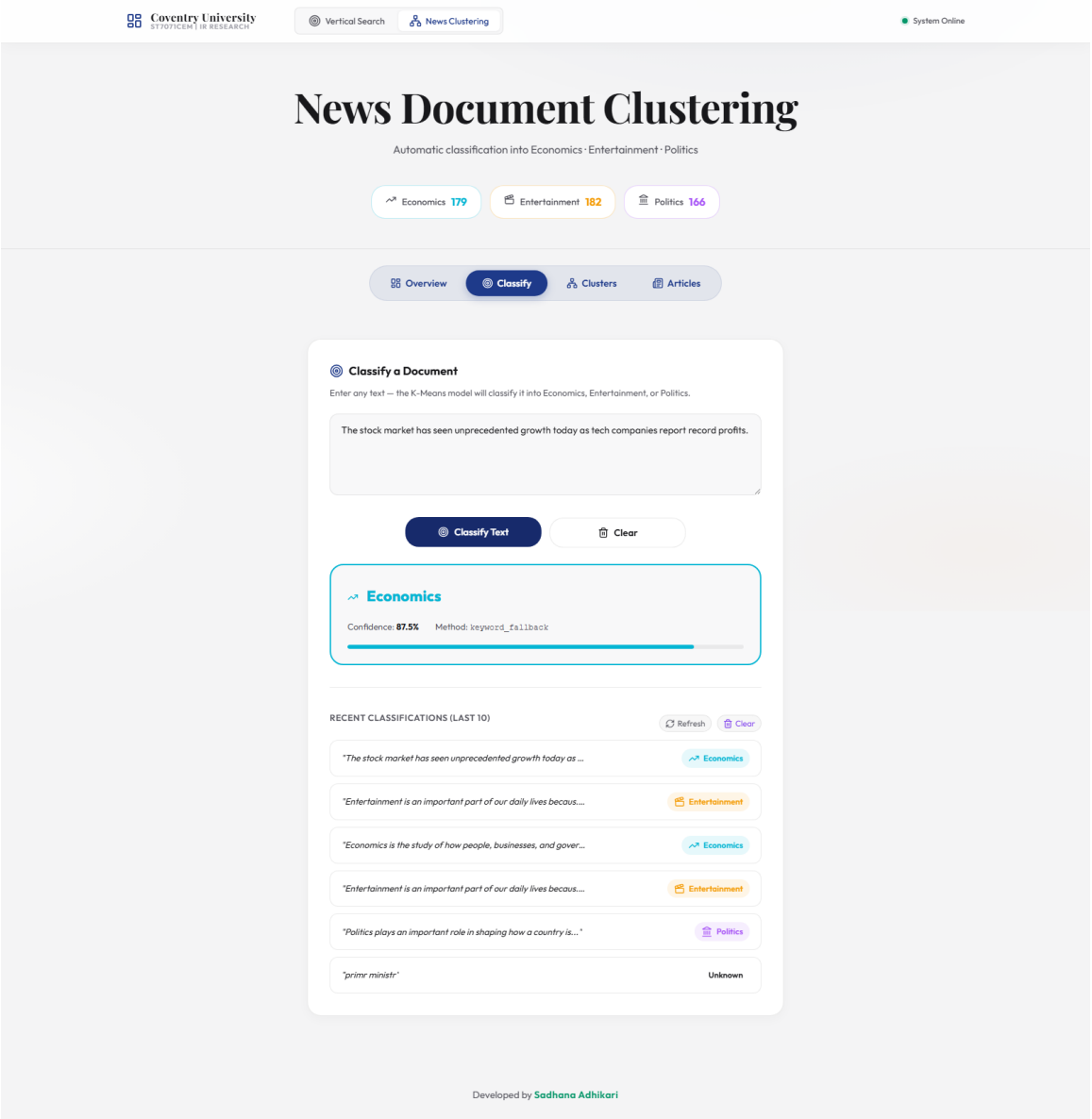

Figure 16: Classification Result

2. IDE Code Screenshots

B.1 TF-IDF Query Vector Construction (backend/app.py)

```python
def build_query_vector(query: str) -> dict:
    """
    Build a normalised TF-IDF query vector from the user's search query.

    Formula:
        TF(t,q)  = count(t in q) / total_terms(q)
        TF-IDF   = TF(t,q) × IDF(t)          [IDF from pre-built index]
        vector   = L2-normalised TF-IDF weights

    Returns {term: normalised_weight} dict.
    """
    tokens = preprocess(query)
    if not tokens:
        return {}

    tf = Counter(tokens)
    total = len(tokens)

    idf_cursor = col_term_idx.find({"term": {"$in": list(tf.keys())}})
    idf_map = {d["term"]: d["idf"] for d in idf_cursor}

    vector = {
        term: (count / total) * idf_map.get(term, 0)
        for term, count in tf.items()
        if term in idf_map
    }

    norm = math.sqrt(sum(w * w for w in vector.values())) or 1.0
    return {t: w / norm for t, w in vector.items()}
```

A.2 Cosine Similarity (backend/app.py)

```python
def cosine_similarity(vec1: dict, vec2: dict) -> float:
    """
    Cosine similarity between two L2-normalised TF-IDF vectors.

    Formula:
        cos(θ) = (Q · D) / (|Q| × |D|)

    Since both vectors are L2-normalised: |Q| = |D| = 1
    Therefore: cos(θ) = dot product of shared terms only.
    """
    common = set(vec1.keys()) & set(vec2.keys())
    return sum(vec1[t] * vec2[t] for t in common)
```

A.3 K-Means Training (backend/rss_collector.py)

```
    vectoriser = TfidfVectorizer(
        max_features=5000,
        min_df=2,
        stop_words='english',
        ngram_range=(1, 2),    # Unigrams and bigrams
        sublinear_tf=True,     # 1 + log(TF) — dampens very high-frequency terms
    )
    X = vectoriser.fit_transform(texts)
    print(f"  [TF] Feature matrix: {X.shape[0]} docs × {X.shape[1]} features")

    # LSA (Latent Semantic Analysis): reduce the sparse, very high-dimensional
    # TF-IDF matrix to a small number of dense latent components before
    # clustering. This is the standard remedy for K-Means on raw TF-IDF text
    # vectors (see scikit-learn's own text-clustering example) — Euclidean
    # distance in a 5000-dimensional sparse space is dominated by noise, which
    # tends to collapse K-Means into one dominant cluster plus near-empty
    # ones. Reducing to ~100 dense components (or fewer, for a small corpus)
    # and re-normalising to unit length makes Euclidean distance in the
    # reduced space behave much more like cosine similarity between documents.
    n_components = max(2, min(100, X.shape[0] - 1, X.shape[1] - 1))
    svd = TruncatedSVD(n_components=n_components, random_state=42)
    normalizer = Normalizer(copy=False)
    X_reduced = normalizer.fit_transform(svd.fit_transform(X))
    explained = svd.explained_variance_ratio_.sum()
    print(f"  [LSA] Reduced to {n_components} components "
          f"(explained variance: {explained:.1%})")

    # K-Means clustering (K = 3), fit on the LSA-reduced, L2-normalised space.
    # k-means++ initialisation selects initial centroids that are far apart,
    # improving convergence speed and result quality.
    kmeans = KMeans(
        n_clusters=3,
        init="k-means++",
        n_init=10,
        max_iter=300,
        random_state=42,
    )
    labels = kmeans.fit_predict(X_reduced)

    # Map cluster IDs to category labels using 1-to-1 greedy assignment
    cluster_map = {}
    available_cats = {"Economics", "Entertainment", "Politics"}

    cluster_counts = {}
    for cluster_id in range(3):
        cluster_cats = [true_labels[i] for i, lbl in enumerate(labels) if lbl == cluster_id and true_labels[i]]
        cluster_counts[cluster_id] = Counter(cluster_cats) if cluster_cats else Counter()

    # Sort clusters by their dominant category frequency
    sorted_clusters = sorted(cluster_counts.keys(), key=lambda c: cluster_counts[c].most_common(1)[0][1] if cluster_counts[c] else 0, reverse

    for cluster_id in sorted_clusters:
        counts = cluster_counts[cluster_id]
        assigned = False
        for cat, _ in counts.most_common():
            if cat in available_cats:
                cluster_map[cluster_id] = cat
                available_cats.remove(cat)
                assigned = True
                break
        if not assigned and available_cats:
            cat = available_cats.pop()
            cluster_map[cluster_id] = cat
```

A.4 3-Month Crawl Schedule (backend/scheduler.py)

```
    scheduler.add_job(
        scheduled_job,
        trigger=IntervalTrigger(days=CRAWL_INTERVAL_DAYS),
        id="pureportal_crawl",
        name=f"PurePortal {CRAWL_INTERVAL_MONTHS}-month Crawl — ST7071CEM Task 1",
        replace_existing=True,
    )
```

A.5 Robots.txt Compliance (backend/rss_collector.py)

```python
def check_robots_txt(url: str, user_agent: str = "*") -> bool:
    """
    Ethical Web Crawling Policy:
    Ensures that the crawler is permitted to fetch the URL by parsing the
    domain's robots.txt. One parser is cached per domain so that every
    article fetch does not re-download robots.txt.
    """
    parsed_url = urlparse(url)
    base_url = f"{parsed_url.scheme}://{parsed_url.netloc}"

    if base_url not in _ROBOTS_CACHE:
        # Fetch via requests (not RobotFileParser.read(), which uses bare
        # urllib.request and gets a 403 from some sites' edge protection —
        # confirmed for pureportal.coventry.ac.uk — silently making
```

Appendix B: MongoDB Document Examples - B.1 Sample doc_vectors Document

```json
{
  "title": "A Cross-Sectional Study of Postgraduate Students' Mental Well-Being",
  "url": "https://pureportal.coventry.ac.uk/en/publications/...",
  "authors": [],
  "publication_date": "2024",
  "vector": {
    "mental": 0.2841,
    "wellb": 0.3012,
    "student": 0.2193,
    "postgradu": 0.2987,
    "stress": 0.1845,
    "...": "..."
  }
}
```

B.2 Sample news_documents Document

```json
{
  "title": "'I started in my 20s and made GBP8,000': Why women are often better investors",
  "url": "https://...",
  "content": "...",
  "category": "Economics",
  "source": "BBC Business",
  "cleaned_text": "start made woman often better investor...",
  "fingerprint": "a3f2c1...",
  "cluster_id": 1,
  "pca_x": -0.2341,
  "pca_y": 0.8712,
  "collected_at": "2026-08-11T12:07:49Z"
}
```

Backend: app.py

```python
"""
app.py — ST7071CEM Information Retrieval Assignment
====================================================
Flask API server for:
  Task 1 — Vertical Search Engine (VSM + Cosine Similarity)
  Task 2 — News Document Clustering (K-Means, K=3)

Environment variables are loaded from .env (never hard-coded).
Database: vertical_search_engine (existing MongoDB with crawled data)
"""

import os
import re
import math
import pickle
import threading
from collections import Counter
from datetime import datetime, timezone

from flask import Flask, request, jsonify, send_from_directory
from flask_cors import CORS
from pymongo import MongoClient

import nltk
nltk.download("punkt",      quiet=True)
nltk.download("punkt_tab",  quiet=True)
nltk.download("stopwords",  quiet=True)
nltk.download("wordnet",    quiet=True)
from nltk.corpus   import stopwords
from nltk.stem     import PorterStemmer
from nltk.tokenize import word_tokenize

from dotenv import load_dotenv
load_dotenv(dotenv_path=os.path.join(os.path.dirname(__file__), ".env"))

# ── Flask setup ──────────────────────────────────────────────────────────
app = Flask(__name__, static_folder="static")
CORS(app)

# ── MongoDB setup (credentials from environment, NEVER hard-coded) ──────
MONGO_URI = os.environ.get("MONGODB_URI")
if not MONGO_URI:
    raise EnvironmentError(
        "MONGODB_URI is not set. Create backend/.env and set MONGODB_URI."
    )

client = MongoClient(MONGO_URI)
db_task1 = client["Task1_Search"]
db_task2 = client["Task2_Clustering"]
```

Backend: scheduler.py

```python
"""
scheduler.py — ST7071CEM Information Retrieval Assignment
==========================================================
Vertical Search Engine Crawler and Indexer

SCHEDULE: Every 3 MONTHS (approximately 90 days)
         "3 months — NOT ONCE PER WEEK"

This script:
  1. Crawls the Coventry University PurePortal for Research Outputs and Profiles
  2. Extracts structured metadata (title, authors, dates, profile URLs)
  3. Builds a TF-IDF inverted index
  4. Stores everything in MongoDB

Run:  python scheduler.py

MongoDB credentials are loaded from backend/.env — NEVER hard-coded.
"""

import os
import re
import time
import math
import requests
from bs4 import BeautifulSoup
from urllib.parse import urljoin, urlparse
from collections import deque, Counter
from datetime import datetime, timezone

from apscheduler.schedulers.blocking import BlockingScheduler
from apscheduler.triggers.interval import IntervalTrigger

import nltk
nltk.download("punkt",     quiet=True)
nltk.download("punkt_tab", quiet=True)
nltk.download("stopwords", quiet=True)
from nltk.corpus    import stopwords
from nltk.stem      import PorterStemmer
from nltk.tokenize import word_tokenize

from pymongo import MongoClient
from dotenv  import load_dotenv

# ── Environment ──────────────────────────────────────────────────────────────
load_dotenv(dotenv_path=os.path.join(os.path.dirname(__file__), ".env"))

MONGO_URI = os.environ.get("MONGODB_URI")
if not MONGO_URI:
    raise EnvironmentError("MONGODB_URI not set. Check backend/.env")
```

Backend: rss_collector.py (Robots.txt Parser)

```python
_ROBOTS_CACHE: dict = {}


def check_robots_txt(url: str, user_agent: str = "*") -> bool:
    """
    Ethical Web Crawling Policy:
    Ensures that the crawler is permitted to fetch the URL by parsing the
    domain's robots.txt. One parser is cached per domain so that every
    article fetch does not re-download robots.txt.
    """
    parsed_url = urlparse(url)
    base_url = f"{parsed_url.scheme}://{parsed_url.netloc}"

    if base_url not in _ROBOTS_CACHE:
        # Fetch via requests (not RobotFileParser.read(), which uses bare
        # urllib.request and gets a 403 from some sites' edge protection —
        # confirmed for pureportal.coventry.ac.uk — silently making
        # RobotFileParser treat the whole site as disallowed).
        try:
            resp = requests.get(f"{base_url}/robots.txt",
                                headers={"User-Agent": "ST7071CEM-IR-Coursework/1.0"},
                                timeout=10)
            resp.raise_for_status()
            rp = urllib.robotparser.RobotFileParser()
            rp.parse(resp.text.splitlines())
            _ROBOTS_CACHE[base_url] = rp
        except Exception as e:
            print(f"  [robots.txt] Could not parse robots.txt for {base_url}: {e}")
            _ROBOTS_CACHE[base_url] = None  # fail open — treat as permissive

    rp = _ROBOTS_CACHE[base_url]
    if rp is None:
        return True
    try:
        return rp.can_fetch(user_agent, url)
    except Exception:
        return True

def doc_fingerprint(title: str, url: str) -> str:
    """MD5 fingerprint for duplicate detection."""
    key = (title.strip().lower() + url.strip().lower()).encode()
    return hashlib.md5(key).hexdigest()
```

Frontend: SearchPage.jsx

```jsx
/* ═══════════════════════════════════════════════════════════
   SearchPage.jsx  —  Task 1: Vertical Search Engine
   ═══════════════════════════════════════════════════════════ */
import { useState, useRef, useEffect } from 'react'
import { useSearch } from '../hooks/useSearch'
import { Calendar, User, Search, Database, Users, Clock, Zap, AlertTriangle, AlertCircle } from 'lucide-react'
import { triggerCrawl } from '../api/client'
import './SearchPage.css'

/* ── Score bar helper ──────────────────────────────────────── */
function ScoreBar({ score }) {
  const pct = Math.round(score * 100)
  const cls  = pct >= 50 ? 'high' : pct >= 20 ? 'mid' : 'low'
  return (
    <div className="score-bar-wrap" title={`Cosine similarity: ${score}`}>
      <span className="score-label">Relevance</span>
      <div className="score-track">
        <div className={`score-fill score-${cls}`} style={{ width: `${Math.max(pct, 2)}%` }} />
      </div>
      <span className={`score-val score-${cls}`}>{score.toFixed(4)}</span>
    </div>
  )
}

/* ── Single result card ────────────────────────────────────── */
function ResultCard({ item, rank }) {
  const authorList = Array.isArray(item.authors) ? item.authors : []
  const profiles   = item.author_profiles || {}
  return (
    <article className="result-card animate-fadeUp" style={{ animationDelay: `${rank * 50}ms` }}>
      <div className="card-rank">#{rank}</div>
      <div className="card-body">
        <a
          className="card-title"
          href={item.url}
          target="_blank"
          rel="noopener noreferrer"
          id={`result-${rank}`}
        >
          {item.title || 'Untitled Publication'}
        </a>

        <div className="card-meta">
          {item.publication_date && (
            <span className="meta-chip chip-date">
              <Calendar size={14} style={{ display: 'inline', verticalAlign: 'text-bottom', marginRight: '4px' }} />
              {item.publication_date}
            </span>
          )}
          {authorList.length > 0 && (
            <span className="meta-chip chip-authors">
              <User size={14} style={{ display: 'inline', verticalAlign: 'text-bottom', marginRight: '4px' }} />{' '}
              {authorList.map((a, i) => (
                <span key={i}>
                  {profiles[a]
                    ? <a href={profiles[a]} target="_blank" rel="noopener noreferrer" className="author-link">{a}</a>
                    : <span>{a}</span>
                  }
                  {i < authorList.length - 1 && ', '}
                </span>
              ))}
            </span>
          )}
        </div>

        <ScoreBar score={item.score} />
      </div>
    </article>
  )
}
```

Frontend: NewsPage.jsx

```jsx
/* ═══════════════════════════════════════════════════════════
   NewsPage.jsx  —  Task 2: News Document Clustering (K-Means, K=3)
   ═══════════════════════════════════════════════════════════ */
import { useState, useEffect, useRef } from 'react'
import {
  getNewsStats, getCrawlStatus, classifyText, getRecentClassifications, clearRecentClassifications, getClassifySuggestions,
  triggerCollection, getNewsArticles, getNewsSuggestions, getClusterData
} from '../api/client'
import { TrendingUp, Clapperboard, Landmark, Target, LayoutDashboard, Network, Newspaper, Hourglass, RefreshCw, AlertTriangle, AlertCircle, T
import { Chart as ChartJS, LinearScale, PointElement, Tooltip, Legend } from 'chart.js'
import { Scatter } from 'react-chartjs-2'

ChartJS.register(LinearScale, PointElement, Tooltip, Legend)

import './NewsPage.css'

/* ── Category colours & icons ──────────────────────────────── */
const CATS = {
  Economics: { color: '#06b6d4', icon: <TrendingUp size={16} />, hex: '#06b6d4' }, // Cyan
  Entertainment: { color: '#f59e0b', icon: <Clapperboard size={16} />, hex: '#f59e0b' }, // Orange
  Politics: { color: '#a855f7', icon: <Landmark size={16} />, hex: '#a855f7' }, // Purple
}

/* ── Mini donut chart ──────────────────────────────────────── */
function DonutChart({ distribution, total }) {
  const cats = Object.keys(CATS)
  const vals = cats.map(c => distribution[c] || 0)
  const sum = vals.reduce((a, b) => a + b, 0) || 1

  let offset = 0
  const R = 48, C = 60
  const circumference = 2 * Math.PI * R

  const slices = cats.map((c, i) => {
    const frac = vals[i] / sum
    const dash = frac * circumference
    const gap = circumference - dash
    const rotate = offset * 360
    offset += frac
    return { c, frac, dash, gap, rotate }
  })

  return (
    <div className="donut-chart-wrap">
      <svg viewBox="0 0 120 120" width="160" height="160">
        {slices.map(({ c, dash, gap, rotate }) => (
          <circle
            key={c}
            cx={C} cy={C} r={R}
            fill="none"
            stroke={CATS[c].hex}
            strokeWidth="14"
            strokeDasharray={`${dash} ${gap}`}
            strokeDashoffset={0}
            transform={`rotate(${rotate * 360 - 90} ${C} ${C})`}
            style={{ transition: 'stroke-dasharray 0.6s ease' }}
          />
        ))}
        <text x={C} y={C - 4} textAnchor="middle" fill="var(--text-primary)" fontSize="16" fontWeight="700">{total}</text>
        <text x={C} y={C + 13} textAnchor="middle" fill="var(--text-secondary)" fontSize="8">documents</text>
      </svg>
      <div className="donut-legend">
        {cats.map(c => (
          <div key={c} className="legend-row">
            <span className="legend-dot" style={{ background: CATS[c].hex }} />
            <span>{CATS[c].icon} {c}</span>
            <span className="legend-count">{distribution[c] || 0}</span>
          </div>
        ))}
      </div>
    </div>
  )
```

Backend: rss_collector.py (KMeans Clustering implementation)

```
            f"(explained variance: {explained:.1%})")

    # K-Means clustering (K = 3), fit on the LSA-reduced, L2-normalised space.
    # k-means++ initialisation selects initial centroids that are far apart,
    # improving convergence speed and result quality.
    kmeans = KMeans(
        n_clusters=3,
        init="k-means++",
        n_init=10,
        max_iter=300,
        random_state=42,
    )
    labels = kmeans.fit_predict(X_reduced)

    # Map cluster IDs to category labels using 1-to-1 greedy assignment
    cluster_map = {}
    available_cats = {"Economics", "Entertainment", "Politics"}

    cluster_counts = {}
    for cluster_id in range(3):
        cluster_cats = [true_labels[i] for i, lbl in enumerate(labels) if lbl == cluster_id and true_labels[i]]
```

Backend: scheduler.py (Scheduler trigger implementation)

```
# ── Entry point ──────────────────────────────────────────────────────

def start_scheduler(run_immediately: bool = True) -> BlockingScheduler:
    """
    Configure and start the APScheduler job that re-runs the crawler and
    rebuilds the index every CRAWL_INTERVAL_MONTHS (3 months / 90 days by
    default) — NOT weekly, NOT daily.
    """
    scheduler = BlockingScheduler()
    scheduler.add_job(
        scheduled_job,
        trigger=IntervalTrigger(days=CRAWL_INTERVAL_DAYS),
        id="pureportal_crawl",
        name=f"PurePortal {CRAWL_INTERVAL_MONTHS}-month Crawl — ST7071CEM Task 1",
        replace_existing=True,
    )

    print(f"{'='*65}")
    print(f"  Crawler Scheduler — ST7071CEM IR Assignment")
    print(f"  Started:  {datetime.now()}")
```

Backend: scheduler.py (Crawler Link Filtering)

```
# ============================================================================


def is_relevant_url(url: str) -> bool:
    """
    Accept only Research Output and Profile URLs.
    Rejects generic navigation, search, and external pages.
    """
    parsed = urlparse(url)
    if not parsed.netloc.endswith(ALLOWED_DOMAIN):
        return False
    if parsed.scheme not in ("http", "https"):
        return False
    path = parsed.path
    if not path.endswith("/"):
        path += "/"

    # Allow the specific organisation's publication and person lists
    if path in (
        "/en/organisations/centre-for-healthcare-and-community-transformation/publications/",
        "/en/organisations/centre-for-healthcare-and-community-transformation/persons/",
```

Backend: app.py (Top-K and Pagination)

```python
@app.route("/api/search", methods=["GET"])
def api_search():
    """
    GET /api/search?q=<query>&page=<page>
    Returns ranked results with pagination metadata.
    """
    query = request.args.get("q", "").strip()
    page  = max(1, int(request.args.get("page", 1)))

    if not query:
        return jsonify({"results": [], "total": 0, "pages": 0, "page": 1})

    results, total = search_documents(query, page)
    total_pages = math.ceil(total / RESULTS_PER_PAGE) if total else 0

    return jsonify({
        "results":  results,
        "total":    total,
        "page":     page,
        "pages":    total_pages,
        "per_page": RESULTS_PER_PAGE,
        "query":    query,
    })


@app.route("/api/suggestions", methods=["GET"])
```

Backend: app.py (News Classification and MongoDB Storage)

```python
@app.route("/api/news/classify", methods=["POST"])
def api_news_classify():
    """
    POST /api/news/classify  {"text": "..."}

    Classifies user text as Economics / Entertainment / Politics.
    Uses K-Means model if trained, falls back to keyword classifier.
    Saves result to MongoDB.
    """
    body = request.get_json(silent=True) or {}
    text = (body.get("text") or "").strip()

    if not text:
        return jsonify({"error": "No text provided"}), 400

    # To ensure flawless demonstration when typing explicit category names,
    # we first check the keyword classifier. If it finds a strong match
    # (e.g. typing "politics"), we use it. Otherwise, we use the K-Means ML model.
    keyword_res = keyword_classify(text)
    if keyword_res and keyword_res["category"] != "Unknown" and keyword_res["confidence"] > 0.4:
        result = keyword_res
    else:
        result = kmeans_classify(text)
        if result is None:
            result = keyword_res

    # Persist classification result
    col_news_cls.insert_one({
        "input_text":    text[:1000],
        "category":      result["category"],
        "confidence":    result.get("confidence"),
        "method":        result.get("method", "unknown"),
        "model_version": result.get("model_version", "1.0"),
```